Data Engineering (2026/27)

Practical Assessments

Índice

1 Description 2
1.1 Schedule 2
1.2 Important 2
1.3 Setup 3
1.4 Repository structure 3
1.5 General Scenario 4
2 Practical Assignments (PA) 4
2.1 PA1 — From raw files to a star schema 4
2.1.1 Scenario 5
2.1.2 Tasks 5
2.1.3 Submit 5
2.2 PA2 — An orchestrated and tested ELT pipeline 5
2.2.1 Scenario 6
2.2.2 Tasks 8
2.2.3 Submit 8
2.3 PA3 — Data protection and tool exploration 8
2.3.1 Scenario 9
2.3.2 Tasks 9
2.3.3 Submit 10

1 Description

You will build one data platform in three steps, in a group, during classes/home. Each step is submitted two days before an individual written test about it. The three assessments together are worth 60% of your final grade; the written final exam is worth 40% (minimum grade 7/20).

- Evaluation
  - ➔ Alternative 1 – (Regular Student, Working Student) (Regular Exam Period – 1st exam)
    - ⇨ Written Final Exam – 40% (Minimum grade required: 7/20)
    - ⇨ Practical Tasks - 60% (Practical Assessments)
      - → Week 6 (19/10/2026) – Submission: 17/10/2026 – 20%
      - → Week 10 (16/11/2026) - Submission: 14/11/2026 – 20%
      - → Week 14 (14/12/2026) - Submission: 12/12/2026 – 20%
  - ➔ Alternative 2 – (Supplementary and September)
    - ⇨ Written Final Exam – 100%

1.1 Schedule

Assessment	Weight	Submit by (23:59)	Individual test
PA1 — Storage formats and analytical modelling	20%	Saturday 17/10/2026	Monday 19/10/2026
PA2 — Orchestrated and tested ELT pipeline	20%	Saturday 14/11/2026	Monday 16/11/2026
PA3 — Data protection and tool exploration	20%	Saturday 12/12/2026	Monday 14/12/2026

1.2 Important

- Groups of **3 students**, fixed for the semester.
- The work is developed during classes, where the professor is available for feedback. It may also be continued outside class.
- Submission: one zip file per group, uploaded to virtual.ipb.pt, with the complete repository and a README.md holding the instructions needed to reproduce the results.
- A demo video (maximum 5 minutes) is submitted together with the zip, before the defense. It is a screen recording with voice that shows the real execution of the work and the outputs it produces; both members of the group speak.
- Late submissions (and in the wrong location) are not accepted.
- The defense is individual and is done on paper, without a laptop. It covers the group's own submission and the concepts behind it, so both members must master all of the work.
- Grading of each assignment:
  - The submitted work is marked from 0 to 20 points, using the criteria of its section (same mark for both members);
  - The defense is marked from 0 to 20 (individual mark). A student with 7 or more (0-20) in the defense receives the mark of the work; a student with less than 7 (0-20) in the defense gets zero for that assignment.
- Grading of each assignment:

- The submitted work is marked from 0 to 20 points, using the criteria of its section (same mark for both members).
- The defense is marked from 0 to 20 (individual mark) and determines how much of the final mark each student receives:

Defense (0–20)	Share of the work mark
17 or more	100%
16	95%
15	90%
14	85%
13	80%
12	75%
11	70%
10	65%
9	60%
8	55%
7	50%
Below 7	0%

1.3 Setup

Create a Python virtual environment and install the course requirements file with `pip`. Airflow (PA2) does not run natively on Windows: use WSL2 or a lab machine.

1.4 Repository structure

Use one repository for the three assessments, with this layout. Folders marked "from PA2" or "PA3" are created when you reach that assessment.

```
project/
├── README.md              # written answers and commands to reproduce, one section per PA
├── requirements.txt
├── .gitignore             # data/source/, data/raw/, *.duckdb, .env
├── architecture/          # diagrams: star schema (PA1), pipeline (PA2)
├── data/
│   ├── source/            # Olist CSV files and the seller JSON; never edit, never commit
│   ├── raw/               # API responses landed by the pipeline, one folder per date (from PA2)
│   ├── staging/           # intermediate files, when used
│   └── curated/           # Parquet exports (PA1) and final outputs
├── sql/                   # profiling.sql, build_model.sql, benchmark queries; erase_customer.sql (PA3)
├── src/                   # Python code: extraction and load (from PA2), helper scripts
├── dbt/                   # from PA2: models, tests, macros
├── airflow/               # from PA2: dags/
├── exploration/           # PA3 Part B: code or notebook, and the exploration log
├── tests/                 # checks you run yourselves outside dbt
└── evidence/              # benchmark table, dbt test output, screenshots, before and after counts
```

1.5 General Scenario

First source: data from OLIST

NorthMart is an online marketplace. Its order data is the public Brazilian E-Commerce Public Dataset by Olist: 99 441 orders and 112 650 order items, purchased between September 2016 and October 2018, in nine CSV files of about 120 MB in total (orders, order items, customers, products, sellers, payments, reviews, geolocation and a category translation table). Download the files from that Kaggle page (a free account is needed) and unzip them into the `data/source/` folder of your repository, keeping the original file names. Do not commit the data files.

Second source: seller profiles in JSON.

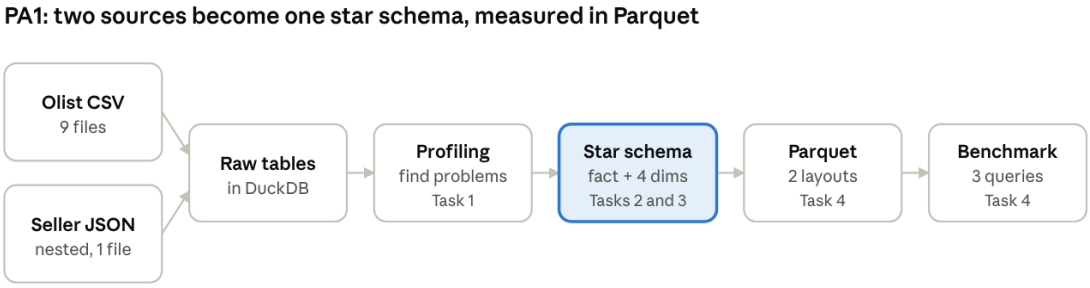
NorthMart's seller management system exports one JSON file, on the course page, with a profile per seller. Put it in `data/source/` next to the CSV files. Unlike the CSV files it is nested, and it was not exported with the same care: expect the kinds of problems a real export has.

```
{
  "seller_id": "d6b664fa0667064c6d76394e7848d790",
  "updated_at": "2018-05-05T08:00:00",
  "profile":  {"tier": "silver", "joined_on": "2016-08-18"},
  "contact":  {"email": "seller_d6b664fa@example.com", "phone": "+55 61 92553-8176"},
  "shipping": {"carriers": ["loggi", "azul_cargo"], "handling_days": 3}
}
```

2 Practical Assignments (PA)

2.1 PA1 — From raw files to a star schema

Turn NorthMart's raw CSV exports and a JSON export of seller profiles into a dimensional model stored in Parquet and queried with DuckDB, and justify each storage decision with measurements.



PA1 flow: 2 sources - 5 steps

Both sources land unchanged in raw tables; everything after that is built from the raw layer. Task 5, the written question, uses the benchmark results.

2.1.1 Scenario

NorthMart's analysts query the CSV exports directly. They complain that reports are slow and that every analyst joins the files differently. You are asked to deliver the first analytical layer.

You receive: the CSV files you downloaded from Kaggle, the JSON file `northmart_seller_profiles.json`, a requirements file (DuckDB, pandas, pyarrow). Create the repository yourselves, with the structure given above.

2.1.2 Tasks

#	Task	Marks
1	Profile the raw files in DuckDB: row counts, keys, null rates and duplicates. Report at least five concrete findings, of which at least two about the JSON file.	3
2	Design a star schema for sales analysis: one fact table at order-item grain and at least four dimensions, including a date dimension. Deliver the diagram and state the grain, the measures and one modelling decision that was not obvious.	6
3	Implement the model as SQL scripts that build the fact and dimension tables in DuckDB from the raw files, with surrogate keys and referential checks. Load the JSON seller profiles, flatten the nested fields, keep one record per seller, and add the tier, the joining date and the handling days to the seller dimension.	4
4	Export the fact table to Parquet, once unpartitioned and once partitioned by a column of your choice. Compare CSV and both Parquet layouts on file size and on the run time of the same three queries: revenue per month over the whole period, revenue and item count for one single month, and the five product categories with the highest revenue.	5
5	Written question (max. 150 words): would NorthMart be better served by a data warehouse, a data lake or a lakehouse? Justify with two facts from your own measurements.	2

2.1.3 Submit

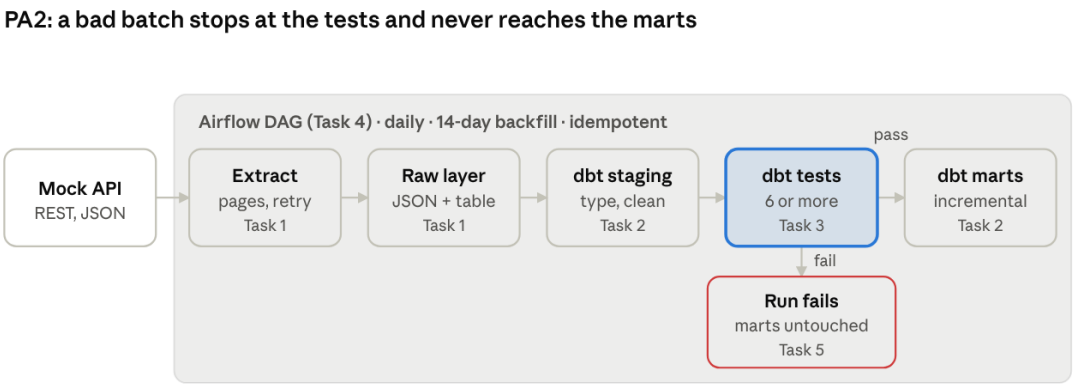
`profiling.sql`, the schema diagram (image or Mermaid), `build_model.sql`, the Parquet outputs, a results table for task 4, and `README.md` with the answers to tasks 1 and 5. Place each file in the folder given in the repository structure.

The model loads and answers the three queries; the grain is stated and defensible; the benchmark is run more than once and the method is described.

Demo video: In at most 5 minutes: run `build_model.sql` from the raw files, show the result of the referential checks, run the three benchmark queries on the three layouts, and show the files produced in `data/curated/`.

2.2 PA2 — An orchestrated and tested ELT pipeline

Replace the manual load of PA1 with a daily pipeline that extracts new orders from a REST API, transforms them with dbt and runs under Airflow, and that stops when the data is wrong.



PA2 pipeline: 5 tasks in one DAG - 1 gate

The tests sit between staging and marts: when they fail, the run stops and the fact table keeps its previous state. Task 6, the written question, uses the task durations of the finished DAG.

2.2.1 Scenario

NorthMart now exposes each day's orders through an internal REST API. The analytical layer must refresh every day without manual work, and a bad batch must never reach the reports.

You receive: a requirements file (Airflow, DuckDB, dbt-duckdb, FastAPI, uvicorn, requests; all versions are pinned), the reference star schema from PA1, a dbt project skeleton, the data file `northmart_api_feed.parquet` (on virtual; put it in `data/source/`) and the script `mock_api.py`, which you run locally with `uvicorn mock_api:app --port 8000` from the root of your repository, so that it finds the source files under `data/`. Use a new DuckDB file for PA2: run the PA1 build script (yours or the reference solution) against it, then delete all rows of the fact table, so that the pipeline fills it.

API contract. `GET http://localhost:8000/orders?date=YYYY-MM-DD&page=N` serves the order items purchased on that date, 100 per page, for the 14 days from 2018-05-01 to 2018-05-14.

```
{
  "date": "2018-05-03",
  "page": 1,
  "total_pages": 4,
  "items": [
    {"order_id": "...", "order_item_id": 1, "customer_id": "...",
     "product_id": "...", "seller_id": "...", "order_status": "delivered",
     "order_purchase_timestamp": "2018-05-03 10:21:07",
     "price": 59.9, "freight_value": 13.2}
  ]
}
```

The API sometimes answers HTTP 503; a retry succeeds. Each day has between 200 and 450 order items, so expect 3 to 5 pages per day. One of the 14 days contains corrupted data. You are not told which. DuckDB accepts one writer at a time: keep this in mind when you run the backfill.

The mock API. Save this as `mock_api.py` in the root of your repository. Treat it as an external service: your pipeline may only talk to it over HTTP and must not read its data file directly.

```
“””NorthMart mock orders API (PA2).

Run from the root of your repository:
    uvicorn mock_api:app –port 8000
Needs fastapi, uvicorn and duckdb, and the file data/source/northmart_api_feed.parquet.
“””
import random
from datetime import date as Date

import duckdb
from fastapi import FastAPI, HTTPException

FEED = “data/source/northmart_api_feed.parquet”
PAGE_SIZE = 100

app = FastAPI(title=”NorthMart orders API”)
con = duckdb.connect()
con.execute(f”CREATE TABLE feed AS SELECT * FROM read_parquet(‘{FEED}’)”)


@app.get(“/orders”)
def orders(date: str, page: int = 1):
    try:
        Date.fromisoformat(date)
    except ValueError:
        raise HTTPException(400, “date must be YYYY-MM-DD”)
    if page < 1:
        raise HTTPException(400, “page starts at 1”)
    if random.random() < 0.1:                       # the service is sometimes unavailable
        raise HTTPException(503, “temporarily unavailable”)

    cur = con.cursor()
    cur.execute(“SELECT * EXCLUDE (seq, day) FROM feed WHERE day = CAST(? AS DATE) ORDER BY seq”,
[date])
    cols = [d[0] for d in cur.description]
    items = [dict(zip(cols, r)) for r in cur.fetchall()]

    total_pages = max(1, -(-len(items) // PAGE_SIZE))
    start = (page – 1) * PAGE_SIZE
    return {“date”: date, “page”: page, “total_pages”: total_pages,
            “items”: items[start:start + PAGE_SIZE]}
```

A date outside the 14 days returns an empty list of items with `total_pages` equal to 1. An invalid date or page returns HTTP 400.

2.2.2 Tasks

#	Task	Marks
1	Extraction: a Python task that calls the API for one execution date, handles pagination and HTTP errors with retries, and lands the raw response unchanged in a `raw` layer.	4
2	Transformation in dbt: staging models that type and clean the raw data, and a mart model that loads the fact table incrementally. The dimensions stay as built in PA1, because the API delivers no new customers, products or sellers.	5
3	Data quality: at least six dbt tests, covering uniqueness, not-null, referential integrity, an accepted range and one custom business rule of your own.	4
4	Orchestration: one Airflow DAG with extract, load, transform and test tasks, a daily schedule, and a backfill of the 14 days. Running the same date twice must not duplicate rows.	4
5	Incident: identify the corrupted day from the test results, show that the DAG failed before the marts were updated, and describe in five lines how the batch should be handled.	2
6	Written question (max. 100 words): name the slowest task of the DAG from the Airflow task durations and propose one optimisation.	1

2.2.3 Submit

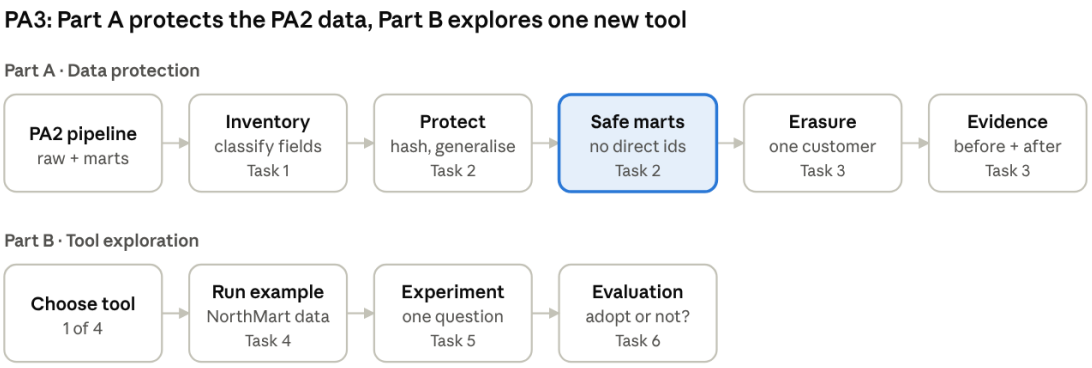
The DAG file, the dbt project, a screenshot of the Airflow grid view after the backfill, the output of `dbt test`, and `README.md` with the answers to tasks 5 and 6. Place each file in the folder given in the repository structure.

The DAG completes for the clean days; an idempotent re-run is demonstrated with row counts before and after; the tests catch every defect of the corrupted day; the custom rule reflects the business and not only the schema.

Demo video (10%). In at most 5 minutes: start the mock API, run the DAG for one clean date and for the corrupted date, show the Airflow grid and the `dbt test` output for both, and show the fact row counts before and after re-running a date.

2.3 PA3 — Data protection and tool exploration

Protect the personal data already in your PA2 pipeline, then explore one emerging tool hands-on and report what you found. Part B is experimentation: you do not build a new pipeline.



PA3: two independent parts - 6 tasks

The two parts are independent and can be done in parallel by the two members, but both must be able to explain both in the test.

2.3.1 Scenario

NorthMart has received its first erasure request under GDPR, and the data protection officer asks for evidence that the analytical layer stores no more personal data than it needs. Separately, the head of data wants a short, evidence-based opinion on one technology the team has never used.

You receive: your own PA2 pipeline (or the reference solution), the identifier of the customer who asked for erasure, and the list of tools below, each with a link to its documentation. Spark and Chroma install with pip; for Kafka and MQTT.

2.3.2 Tasks

#	Task	Marks
	Part A — Data protection	
1	Personal data inventory: classify every field of the raw orders, customers and seller profiles as identifier, quasi-identifier or non-personal, and state the purpose and retention period for each one you keep.	3
2	Protection in the pipeline: in the dbt models, anonymize the customer identifier with a keyed hash whose key is read from an environment variable, and anonymize the customer location. Free-text fields such as review comments must not be loaded into the marts at all. Explain why the result is anonymized and not anonymized.	4
3	Erasure: write and run the procedure that removes the given customer from the marts and from the raw layer (raw tables, files landed in data/raw/ and Parquet exports in data/curated/), and show evidence before and after.	3
	Part B — Tool exploration	
4	Get your chosen tool running and reproduce one minimal working example on NorthMart data. Keep a	4

#	Task	Marks
	short log of what you tried, what failed, and how you solved it.	
5	Run one small experiment that answers a question of your choice, and report the result with its numbers or outputs.	3
6	Evaluation (max. 300 words): what the tool is for, when it should not be used, what it costs in complexity, and whether NorthMart should adopt it now.	3

Tools for Part B. Choose one; at most three groups per tool, first come first served.

Tool	Example of a question to answer
Apache Spark (PySpark, local mode)	Is the same aggregation faster in Spark or in DuckDB at this data size, and why?
Apache Kafka	What happens to the messages when a consumer stops and restarts?
MQTT, with Apache NiFi or a Python client	How do the three quality-of-service levels differ in what is delivered?
Chroma (vector database) for retrieval	Does the default embedding model retrieve the relevant reviews when they are written in Portuguese?

2.3.3 Submit

the inventory table, the changed dbt models, the erasure script with its before and after output, the exploration code or notebook with its log, and `README.md` with the answers to tasks 2, 5 and 6. Place each file in the folder given in the repository structure.

In Part A the hashing key is kept outside the repository and the customer is removed from every layer. In Part B the tool does not have to end up working perfectly: what counts is an experiment with a clear question, honest results and a recommendation argued as a trade-off.

Demo video (10%). In at most 5 minutes: show the protected dimension, run the erasure script and show the before and after counts, then run your tool's working example and your experiment and show their outputs.